

CS2310: OPERATING SYSTEMS

Group	18
Class	SY Computer Engineering A
Batch	3
PRN	12411297

PHASE 1: A MULTIPROGRAMMING OPERATING SYSTEM

PROBLEM STATEMENT

To implement phase 1 of an operating system, including the following:

1. CPU/Machine simulation
2. Supervisor call through interrupt

CODE

```
import java.util.*;
import java.io.*;

public class phase1{
    //4 registers in cpu
    public static char[] R = new char[4];
    public static char[] IR = new char[4];
    public static int IC;
    public static boolean C;
    public static int SI;

    //memory
    public static char[][] mem = new char[100][4];

    public static int numinstr; //no. of instrs in prog card
    public static char[] buffer = new char[40];
    public static Scanner sc;
```

```

public static void main(String[] args) throws Exception{
    load();
}

public static void load() throws Exception{
    String line, word;
    int lineptr = 0;
    int instcnt = 0;
    File f = new File("input.txt");
    sc = new Scanner(f);

    while(sc.hasNextLine()){
        line = sc.nextLine();
        if(line.length()>4) word = line.substring(0,4);
        else word = line;

        if(word.equals("$AMJ")){
            lineptr = 0; instcnt = 0;
            numinstr = Integer.parseInt(line.substring(8,12));
            init();
        }
        else if(word.equals("$DTA")) executeprog();
        else if(word.equals("$END")) {displaymem(); continue;}
        else{
            line = line.trim();
            char[] instructions = line.toCharArray();

            for(int i = 0; i<instructions.length; i++) buffer[i] =
instructions[i];

            for(int i = 0; i<instructions.length; i++){
                if(instcnt != 0 && instcnt%4 == 0) lineptr++;

                if(instructions[i] == 'H') {mem[lineptr][0] = 'H';
instcnt = ((instcnt/4)+1)*4;}
                else {mem[lineptr][instcnt%4] = buffer[i]; instcnt++;}
            }

            for(int i = 0; i<40; i++) buffer[i] = '-';
        }
    }
}

```



```

        SI = 2; IC++;
        mastermode(sc);
        break;

    case "LR":
        IC++;
        temp = "" + IR[2] + IR[3];
        addr = Integer.parseInt(temp);
        for(int i = 0; i<4; i++) R[i] = mem[addr][i];
        break;

    case "SR":
        IC++;
        temp = "" + IR[2] + IR[3];
        addr = Integer.parseInt(temp);
        for(int i = 0; i<4; i++) mem[addr][i] = R[i];
        break;

    case "CR":
        IC++;
        temp = "" + IR[2] + IR[3];
        addr = Integer.parseInt(temp);
        for(int i = 0; i<4; i++){
            if(R[i] == mem[addr][i]) C = true;
            else {C = false; break;}
        }
        break;

    case "BT":
        if(C==true) {
            temp = "" + IR[2] + IR[3];
            addr = Integer.parseInt(temp);
            IC = addr;
        }
        else IC++;
        break;

        default: System.out.println("Invalid instruction
detected: " + opcode + "\nExiting.");
    }
}

```

```

    }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void mastermode(Scanner sc) throws Exception{
    switch(SI){
        case 1: read(sc); break;
        case 2: write(); break;
        case 3: terminate(); break;
        default: System.out.println("Invalid system interrupt: " + SI
+ "\nExiting.");
    }
}

public static void read(Scanner sc){
    String data = sc.nextLine();
    char[] chars = data.toCharArray();
    int len = data.length();
    int loc = Integer.parseInt(""+IR[2]+IR[3]);
    int i = 0, m = 0;

    while(i<len){
        if(i%4 == 0 && m!=0) loc++;
        mem[loc][m%4] = chars[i];
        i++; m++;
    }
}

public static void write() throws Exception{
    int loc = Integer.parseInt(""+IR[2]+IR[3]);
    String data = "";

    for(int i = loc; i<loc+10; i++){
        for(int j = 0; j<4; j++){
            if(mem[i][j] != '-') data += mem[i][j];
            else data += ' ';
        }
    }
}

```

```

        FileWriter fw = new FileWriter("output.txt", true);
        data = data.trim() + "\n";
        fw.write(data);
        fw.close();
    }

    public static void displaymem() {
        for(int i=0; i<100; i++){
            if(i%10==0 && i!=0){

System.out.println("-----");
-----");
                }

                System.out.println(i+"    "+mem[i][0]+mem[i][1]+mem[i][2]+mem[i][3]);
            }

            System.out.println();
            System.out.println();
            System.out.println();
        }

        public static void terminate() throws Exception{
            FileWriter fw = new FileWriter("output.txt", true);
            fw.write("\n\n");
            fw.close();
        }
    }
}

```

INPUT AND OUTPUT FILES

```

(muggloaf@alesha)-[~/phase1]
$ cat input.txt
$AMJ000100030001
GD10PD10H
$DTA
Hello World!
$END0001
$AMJ000200110001
GD20GD30GD40GD50LR20CR30BT09PD50HPD40
$DTA
VIT
VIIT
SAME
NOT SAME
$END0002
$AMJ000300190004
GD20LR20SR30SR31SR32SR33SR40SR43SR50SR53
SR60SR61SR62SR63PD30PD40PD50PD60H
$DTA
*
$END0003
$AMJ000400280001
GD30GD40GD50LR20CR50BT12CR51BT16CR52BT20
CR53BT24LR40SR51PD50HLR40SR52PD50H
LR40SR53PD50HLR40SR54PD50H
$DTA
car
cat
The car purrs
$END0004
$AMJ000400270001
GD30GD40LR30CR40BT11CR41BT15CR42BT19CR43
BT23LR50SR40PD40HLR50SR41PD40HLR50
SR42PD40HLR50SR43PD40H
$DTA
The
The apple is red
$END0005

```

```

(muggloaf@alesha)-[~/phase1]
$ cat output.txt
Hello World!

NOT SAME

*   *   *   *
*           *
*           *
*   *   *   *

The cat purrs

apple is red

```

SYSTEM SETUP AND INSTRUCTIONS

This implementation of a multiprogramming operating system consists of the CPU and memory. The basic unit of the memory is a word of 4 bytes. The system's memory is of 400 bytes, and it is divided into 10 blocks of 10 words each. The CPU consists of 4 main parts, which are:

1. General Purpose Register (R): 4 bytes
2. Instruction Register (IR): 4 bytes
3. Instruction Pointer (IP): 2 bytes
4. Toggle bit (C): 1 byte

Along with this, there is also a flag for System Interrupt (SI). A system interrupt occurs when we need to perform I/O operations or end a program. Lastly, we have a buffer register of 40 bytes, i.e 1 block, which temporarily holds instructions as they are loaded into the memory.

Speaking of instructions, the instruction set consists of 7 instructions only. These are:

1. Get Data (GDxx): This is the instruction to get data from the user and store it in the memory. xx denotes a memory location (00-99) where the data will be stored.
2. Print Data (PDxx): This is the instruction to fetch data from the memory.
3. Load Register (LRxx): This is the instruction to load R with a value from the memory.
4. Store Register (SRxx): This instruction stores the data currently stored in R to memory location xx.
5. Compare Register (CRxx): This instruction compares the value in R with the memory location xx. If the two values are equal, the toggle bit C is set to true. Else it is false.
6. Branch if True (BTxx): It checks for a condition and moves the control to the instruction at location xx if the condition is true.
7. Halt (H): This instruction marks the end of the execution of the current program. No instructions will be executed after it.

Programs are fed to the OS through job cards. A card reader reads their contents and a line printer prints the output. In our simulation, this is being done through 2 .txt files. The input file consists of 5 jobs. Every job consists of 3 main parts:

- Control cards: These are used to instruct the CPU on what to do. These are of 3 types:
 - \$AMJ: AMJ (stands for A Multiprogramming Job) marks the start of a new job. It is followed by 12 bits, the first 4 of which are the job ID, the next 4 are the number of time units required for program execution and the last 4 are the number of lines the output should be.
 - \$DTA: This control card denotes that the instructions have been loaded in the memory and that the next line of the job will be the data.
 - \$END: This control card marks the end of the job. It is followed by the job ID.
- Program cards: These contain the instructions that will be executed during program execution.
- Data cards: These contain the actual data for the program.

We always start reading the instructions and data from program location 00. GD and PD are block instructions, i.e they operate on entire blocks at once. The rest of the instructions operate on single memory locations.

FUNCTION DESCRIPTION

1. `main()` - This is the entry point to the program. Its only purpose is to call the `load()` function to begin the program.
2. `load()` - This function is used to start reading the input file. It reads the file line by line and checks for the following:
 - a. If the line starts with `$AMJ` - A new program has begun. Call `init()` to initialize the memory and registers.
 - b. If the line starts with `$DTA` - Programs have been loaded, and the data follows. Call `executeprog()` to start the program execution.
 - c. If the line starts with `$END` - The program is over. Call `displaymem()` to display the memory contents.
 - d. If the line starts with anything else, it must surely be the program instructions. Load the instructions in the memory.
3. `init()` - This function clears all the values in the memory and registers and replaces them with our chosen empty byte character, '-' for the next program.
4. `executeprogram()` - This function reads the opcode of the instructions stored and executes them accordingly. It uses a switch block to match the opcode and depending on the opcode, it does the following:
 - a. GD, PD, H: Since these instructions trigger the system interrupt, it sets the SI bit to 1, 2, 3 respectively and calls the `mastermode()` function to switch to master mode to handle the interrupt.
 - b. LR, SR, CR: It parses the address given in the command and accordingly sets the value of the register or memory location or compares values and sets C.
5. `mastermode(Scanner sc)` - Depending on the value of SI, it calls the `read(sc)`, `write()`, or `terminate()` functions.
6. `read(Scanner sc)` - It moves to the next line of the file and reads the data from the data card into the memory at the specified location.
7. `write()` - It writes the output to the output file.
8. `displaymem()` - It displays the contents of the memory.
9. `terminate()` - It adds two new lines to the output file to separate the outputs of two jobs.

SPOTLIGHT ON EACH JOB

1. *Print “Hello World!”*

Job:

\$AMJ000100030001
GD10PD10H
\$DTA
Hello World!
\$END0001

Output:

Hello World!

Memory:

0	GD10
1	PD10
2	H---
3	----
4	----
5	----
6	----
7	----
8	----
9	----

10	He ll
11	o Wo
12	rld!
13	----
14	----
15	----
16	----
17	----
18	----
19	----

20	----
21	----
22	----
23	----
24	----
25	----
26	----
27	----
28	----
29	----

30	----
31	----
32	----
33	----
34	----
35	----
36	----
37	----
38	----
39	----

40	----
41	----
42	----
43	----
44	----
45	----
46	----
47	----
48	----
49	----

50	----
51	----
52	----
53	----
54	----
55	----
56	----
57	----
58	----
59	----

60	----
61	----
62	----
63	----
64	----
65	----
66	----
67	----
68	----
69	----

70	----
71	----
72	----
73	----
74	----
75	----
76	----
77	----
78	----
79	----

80	----
81	----
82	----
83	----
84	----
85	----
86	----
87	----
88	----
89	----

90	----
91	----
92	----
93	----
94	----
95	----
96	----
97	----
98	----
99	----

2. Check if two strings are the same (check if VIT and VIIT are the same)

Job:

\$END0001
\$AMJ000200110001
GD20GD30GD40GD50LR20CR30BT09PD50HPD40
\$DTA
VIT
VIIT
SAME
NOT SAME
\$END0002

Output:

NOT SAME

Memory:

0	GD20
1	GD30
2	GD40
3	GD50
4	LR20
5	CR30
6	BT09
7	PD50
8	H---
9	PD40

10	----
11	----
12	----
13	----
14	----
15	----
16	----
17	----
18	----
19	----

20	VIT-
21	----
22	----
23	----
24	----
25	----
26	----
27	----
28	----
29	----

30	VIIT
31	----
32	----
33	----
34	----
35	----
36	----
37	----
38	----
39	----

40	SAME
41	----
42	----
43	----
44	----
45	----
46	----
47	----
48	----
49	----

50	NOT
51	SAME
52	----
53	----
54	----
55	----
56	----
57	----
58	----
59	----

60	----
61	----
62	----
63	----
64	----
65	----
66	----
67	----
68	----
69	----

70	----
71	----
72	----
73	----
74	----
75	----
76	----
77	----
78	----
79	----

80	----
81	----
82	----
83	----
84	----
85	----
86	----
87	----
88	----
89	----

90	----
91	----
92	----
93	----
94	----
95	----
96	----
97	----
98	----
99	----

3. *Print a rectangle pattern with '*'*

Job:

```
$AMJ000300190004
GD20LR20SR30SR31SR32SR33SR40SR43SR50SR53
SR60SR61SR62SR63PD30PD40PD50PD60H
$DTA
*
$END0003
```

Output:

```
*   *   *   *
*           *
*           *
*   *   *   *
```

Memory:

```
0      GD20
1      LR20
2      SR30
3      SR31
4      SR32
5      SR33
6      SR40
7      SR43
8      SR50
9      SR53
-----
10     SR60
11     SR61
12     SR62
13     SR63
14     PD30
15     PD40
16     PD50
17     PD60
18     H---
19     ----
-----
20     *---
21     ----
22     ----
23     ----
24     ----
25     ----
26     ----
27     ----
28     ----
29     ----
-----
30     *---
31     *---
32     *---
33     *---
34     ----
35     ----
36     ----
37     ----
38     ----
39     ----
-----
40     *---
41     ----
42     ----
43     *---
44     ----
45     ----
46     ----
47     ----
48     ----
49     ----
```

```
-----
50     *---
51     ----
52     ----
53     *---
54     ----
55     ----
56     ----
57     ----
58     ----
59     ----
-----
60     *---
61     *---
62     *---
63     *---
64     ----
65     ----
66     ----
67     ----
68     ----
69     ----
-----
70     ----
71     ----
72     ----
73     ----
74     ----
75     ----
76     ----
77     ----
78     ----
79     ----
-----
80     ----
81     ----
82     ----
83     ----
84     ----
85     ----
86     ----
87     ----
88     ----
89     ----
-----
90     ----
91     ----
92     ----
93     ----
94     ----
95     ----
96     ----
97     ----
98     ----
99     ----
```

4. Replace one string with another (Replace “car” with “cat”)

Job:

\$AMJ000400280001
GD30GD40GD50LR30CR50BT12CR51BT16CR52BT20
CR53BT24LR40SR51PD50HLR40SR52PD50H
LR40SR53PD50HLR40SR54PD50H
\$DTA
car
cat
The car purrs
\$END0004

Output:

The cat purrs

Memory:

0	GD30	50	The
1	GD40	51	cat-
2	GD50	52	purr
3	LR30	53	s---
4	CR50	54	----
5	BT12	55	----
6	CR51	56	----
7	BT16	57	----
8	CR52	58	----
9	BT20	59	----
10	CR53	60	----
11	BT24	61	----
12	LR40	62	----
13	SR51	63	----
14	PD50	64	----
15	H---	65	----
16	LR40	66	----
17	SR52	67	----
18	PD50	68	----
19	H---	69	----
20	LR40	70	----
21	SR53	71	----
22	PD50	72	----
23	H---	73	----
24	LR40	74	----
25	SR54	75	----
26	PD50	76	----
27	H---	77	----
28	----	78	----
29	----	79	----
30	car-	80	----
31	----	81	----
32	----	82	----
33	----	83	----
34	----	84	----
35	----	85	----
36	----	86	----
37	----	87	----
38	----	88	----
39	----	89	----
40	cat-	90	----
41	----	91	----
42	----	92	----
43	----	93	----
44	----	94	----
45	----	95	----
46	----	96	----
47	----	97	----
48	----	98	----
49	----	99	----

5. Remove the article "The" from a sentence

Job:

\$AMJ000400270001
GD30GD40LR30CR40BT11CR41BT15CR42BT19CR43
BT23LR50SR40PD40HLR50SR41PD40HLR50
SR42PD40HLR50SR43PD40H
\$DTA
The
The apple is red
\$END0005

Output:

apple is red

Memory:

0	GD30
1	GD40
2	LR30
3	CR40
4	BT11
5	CR41
6	BT15
7	CR42
8	BT19
9	CR43

10	BT23
11	LR50
12	SR40
13	PD40
14	H---
15	LR50
16	SR41
17	PD40
18	H---
19	LR50

20	SR42
21	PD40
22	H---
23	LR50
24	SR43
25	PD40
26	H---
27	----
28	----
29	----

30	The-
31	----
32	----
33	----
34	----
35	----
36	----
37	----
38	----
39	----

40	----
41	appl
42	e is
43	red
44	----
45	----
46	----
47	----
48	----
49	----

50	----
51	----
52	----
53	----
54	----
55	----
56	----
57	----
58	----
59	----

60	----
61	----
62	----
63	----
64	----
65	----
66	----
67	----
68	----
69	----

70	----
71	----
72	----
73	----
74	----
75	----
76	----
77	----
78	----
79	----

80	----
81	----
82	----
83	----
84	----
85	----
86	----
87	----
88	----
89	----

90	----
91	----
92	----
93	----
94	----
95	----
96	----
97	----
98	----
99	----

CONCLUSION

To conclude, in phase 1 of creating a multiprogramming operating system, we have implemented a 100-word memory with functioning CPU registers. Only one type of interrupt, i.e. the system interrupt has been implemented. Programs are executed one at a time. An instruction set comprising 7 instructions is used to carry out the program execution. The system is functional under the assumptions that the entered output caters to the code written and contains no errors. 5 jobs were written and carried out with no errors. Thus, phase 1 is complete.